

# Lab05: TCP congestion control

1. Start up your web browser. Go the <http://gaia.cs.umass.edu/wiresharklabs/alice.txt> and retrieve an ASCII copy of Alice in Wonderland. Store this file somewhere on your computer.

alice.txt

2. Next go to <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
3. Use the 'Browse' button to enter the name of the file (full path name) on your computer containing Alice in Wonderland (or do so manually). Don't yet press the "Upload alice.txt file" button.
4. Now start up Wireshark and begin packet capture (Capture→Start) and then press OK on the Wireshark Packet Capture Options screen (we'll not need to select any options here).
5. Returning to your browser, press the "Upload alice.txt file" button to upload the file to the [gaia.cs.umass.edu](http://gaia.cs.umass.edu) server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
6. Stop Wireshark packet capture.
7. Filter the packets displayed in the Wireshark window by entering "tcp" (lowercase)

- 
1. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to [gaia.cs.umass.edu](http://gaia.cs.umass.edu)?
-

```
Ethernet II, Src: Intel_0f:83:63 (08:9d:f4:0f:83:63), Dst: Secuicom c8:28:6a (00:05:66:c8:28:6a)
Internet Protocol Version 4, Src: 172.17.108.88, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 44952
  Destination Port: 80
  [Stream index: 25]
  [Stream Packet Number: 1]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1568581946
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 ..... = Header Length: 32 bytes (8)
  Flags: 0x002 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    ....0... = Congestion Window Reduced: Not set
    ....0... = ECH-Echo: Not set
    ....0... = Urgent: Not set
    ....0... = Acknowledgment: Not set
    ....0... = Push: Not set
    ....0... = Reset: Not set
    .....1.. = Syn: Set
    .....0.. = Fin: Not set
```

이 Initiate 패킷을 보면,

나의 IP Source Address: 172.17.108.88 임을 알 수 있다.

또한 Source Port: 44952 가 사용되었음을 확인했다.

2. What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?

```
C:\Users\nneighbor>nslookup gaia.cs.umass.edu
서버: ns.lgtelecom.com
Address: 164.124.101.2
권한 없는 응답:
이름: gaia.cs.umass.edu
Address: 128.119.245.12
```

nslookup을 통해 확인한 gaia.cs.umass.edu의 IP주소는 임을 확인했고, 128.119.245.12 이는 1번 질문에서 확인한 Destination Address 128.119.245.12와 일치한다.

3. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in the segment that identifies the segment as a SYN segment?

```

Ethernet II, Src: Intel_0f:83:63 (08:9d:f4:0f:83:63), Dst: Secuicom c8:28:6a (00:05:66:c8:28:6a)
Internet Protocol Version 4, Src: 172.17.108.88, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 44952
  Destination Port: 80
  [Stream Index: 25]
  [Stream Packet Number: 1]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1568581946
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  Flags: 0x002 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...0 = Acknowledgment: Not set
    .... .... = Push: Not set
    .... ..0. = Reset: Not set
    .... ...1 = Syn: Set
    .... .... = Fin: Not set

```

이 패킷은

172.17.108.88:44952 → 128.119.245.12:80 인 것을 보아 호스트(나)로부터 출발했고,

응답임을 알 수 있는 ACK 플래그가 '.... ..0 .... = Acknowledgment: Not set' 으로 0이며

시퀀스 번호를 동기화하자는 요청인 SYN(Synchronize sequence numbers) 플래그가 '.... .... ..1. = Syn: Set'으로 1 이므로 연결 시작을 제안하는 첫번째 SYN 패킷임을 알 수 있다.

4. What is the sequence number of the SYNACK segment sent by [gaia.cs.umass.edu](http://gaia.cs.umass.edu) to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the SYNACK segment? How did [gaia.cs.umass.edu](http://gaia.cs.umass.edu) determine that value? What is it in the segment that identifies the segment as a SYNACK segment?

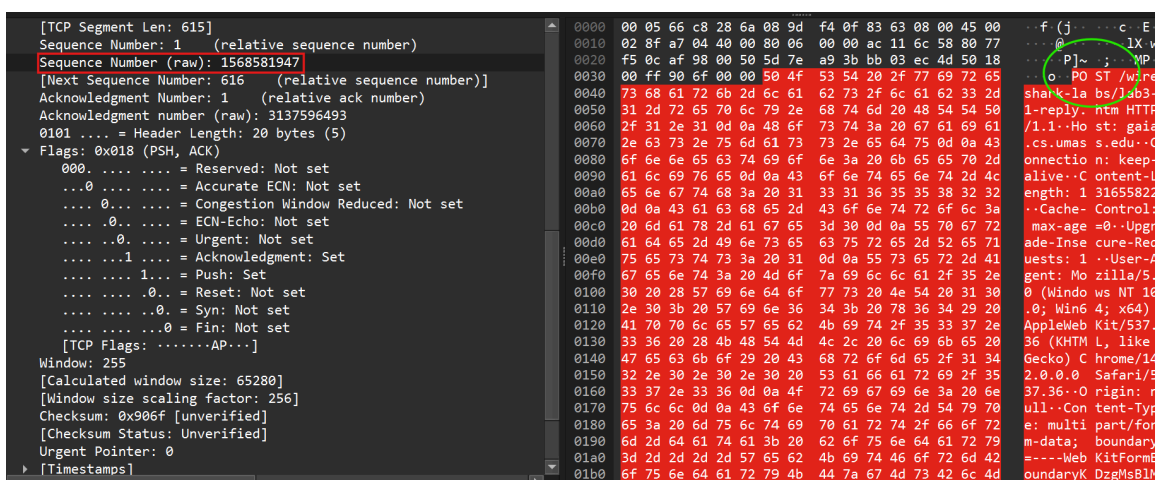
```

Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 0
  Source Port: 80
  Destination Port: 44952
  [Stream index: 25]
  [Stream Packet Number: 2]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3137596492
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1568581947
  1000 .... = Header Length: 32 bytes (8)
  Flags: 0x012 (SYN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 = Acknowledgment: Set
    .... .... = Push: Not set
    .... ..0. = Reset: Not set
    .... ...1 = Syn: Set
    .... .... = Fin: Not set
  [TCP Flags: .....A..S.]

```

- 클라이언트의 initiative SYN 패킷에 대한 응답 SYN\_ACK 패킷의 Sequence number는 raw 기준 '3137596492' 이다.
- SYN\_ACK 패킷의 Acknowledgment number (raw)는 '1568581947' 이다.
- 클라이언트의 initiative SYN 패킷에 대한 응답 SYN\_ACK 패킷의 Sequence number는 host가 독립적으로, 랜덤하게 결정한다.
- SYN\_ACK 패킷의 Acknowledgment number (raw)는 '1568581947'는 Client의 SYN 패킷 Sequence Number (raw) '1568581946' 의 응답으로, 데이터 없는 단순 응답이니 마지막 byte에 1byte 소비(+1) 하여 정해진다.
- SYN\_ACK 패킷의 Acknowledgment number (raw) '1568581947'은 Client의 SYN 패킷 Sequence Number (raw)에 1이 더해진 것으로 관찰되고, flag 값이 SYN + ACK 이며, Src와 Dst 값이 Src: 128.119.245.12, Dst: 172.17.108.88 으로 Client의 SYN 패킷과 정반대이다. 따라서 이 패킷은 SYN\_ACK 패킷이다.

5. What is the sequence number of the TCP segment containing the HTTP POST command? Note that in order to find the POST command, you'll need to dig into the packet content field at the bottom of the Wireshark window, looking for a segment with a "POST" within its DATA field.



5341번 51.672244 172.17.108.88 128.119.245.12 TCP 669 44952 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=615 [TCP PDU reassembled in 51109]

이 패킷이 [PSH, ACK] 라고 표시되어있어, packet content field를 파헤쳐 보니, 'POST'라고 작성되어있음을 확인했다. 따라서 이 세그먼트가 HTTP POST 요청의 시작(혹은 일부)을 포함하고 있음을 알 수 있다.

이 패킷의 Sequence Number (raw)는 '1568581947'이며, 이는 클라이언트 기준  $1568581946 + 1 = 1568581947$ 으로, 3-way handshake의 세 번째 단계에서 클라이언트가 서버로 보낸 ACK가 함께 실린(piggybacked) HTTP POST 요청 세그먼트이다. 즉, 클라이언트 기준으로는 3-way handshake 절차가 완료된 직후의 첫 번째 애플리케이션 계층 데이터 전송(HTTP 요청)이다.

- Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection.

1. What are the sequence numbers of the first six segments in the TCP connection (including the segment containing the HTTP POST)?

1. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0

Sequence Number (raw): 1568581946

2. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 0, Ack: 1, Len: 0

Sequence Number (raw): 3137596492

3. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 0

Sequence Number (raw): 1568581947

4. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 615

Sequence Number (raw): 1568581947 (segment containing the HTTP POST)

5. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 616, Ack: 1, Len: 13140

Sequence Number (raw): 1568582562

6. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 1, Ack: 616, Len: 0

Sequence Number (raw): 3137596493

2. At what time was each segment sent? When was the ACK for each segment received?

1. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0  
(Client SYN, first handshake)sent time: 51.469091 , ACK receive time: 51.671777 (from Host SYN\_ACK, second handshake)
2. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 0, Ack: 1, Len: 0  
(Host SYN\_ACK, second handshake)sent time: 51.671777 , ACK receive time: 51.671842 (from Client ACK, third handshake)
3. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 0  
(Client ACK, third handshake)sent time: 51.671842 , ACK receive time: 51.876959 (from Host, cumulative ACK, 6th segment)
4. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 615  
(Client, first data segment including HTTP POST)sent time: 51.672244 , ACK receive time: 51.876959 (from Host, cumulative ACK, 6th segment)
5. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 616, Ack: 1, Len: 13140  
(Client, rest massive data segment)sent time: 51.675179 , ACK receive time: 51.876959 (from Host, cumulative ACK, 7th segment)
6. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 1, Ack: 616, Len: 0  
(Host, cumulative ACK)sent time: 51.876959, ACK receive time: 51.877007(from Client, cumulative ACK, 10th segment)

이를 작성하는 도중,

1. 클라이언트의 핸드셰이크 요청 SYN
2. 호스트의 '클라이언트의 1번 SYN'에 대한 SYN\_ACK
3. 클라이언트의 '호스트의 SYN\_ACK에 대한 ACK'가 담긴 HTTP POST 요청
4. 클라이언트의 첫번째 데이터 세그먼트 전송(3번의 2번에 대한 ACK 값 유지)
5. 클라이언트의 두번째, 대량 데이터 세그먼트 전송(이더넷 gate에서 분할됨)

6. 3,4번에 대한 호스트의 cumulative ack
7. 이더넷 단에서 쪼개져 전송된 5번 세그먼트 중 첫번째에 대한 호스트의 cumulative ack
8. 이더넷 단에서 쪼개져 전송된 5번 세그먼트 중 두번째에 대한 호스트의 cumulative ack
9. 이더넷 단에서 쪼개져 전송된 5번 세그먼트 중 세번째에 대한 호스트의 cumulative ack
10. 클라이언트의 '호스트의 6,7,8,9 세그먼트에 대한 ack'가 담긴 두번째 대량 데이터 전송

의 흐름으로 구성되어있음을 확인했다.

특히 4번 세그먼트에서 클라이언트가 3번 세그먼트의  $ack = 1$  을 유지한 것을 보고, 3번과 4번의 Acknowledgment number (raw)를 비교했더니, '3137596493' 으로 실제로 같았다.

새로운 ACK 대상이 없으면 ACK 필드를 0으로 비워둘거라 한 생각이 깨졌으며, 수업시간에 배웠던 대로 '상대방으로부터의 다음 세그먼트 기댓값을 주려면 ACK num을 전달해야 하고, 이에 필드값도  $ACK = 1$ 로 채워지겠구나' 하는 것을 깨달았다.

즉, TCP의 ACK 비트  $ACK = 1$ 은, 이 세그먼트에 유효한 ACK num이 있다는 뜻이고, 그 값은 상대방이 보내야 할 다음 시퀀스 번호이다.

3. Given the difference between when each TCP segment was sent, and when its acknowledgement was received, what is the RTT value for each of the six segments?

#### D

1. 세그먼트 1

$$51.671777 - 51.469091 = 0.202686 \text{ 초}$$

2. 세그먼트 2

$$51.671842 - 51.671777 = 0.000065 \text{ 초}$$

3. 세그먼트 3

$$51.876959 - 51.671842 = 0.205117 \text{ 초}$$

4. 세그먼트 4

$$51.876959 - 51.672244 = 0.204715 \text{ 초}$$

##### 5. 세그먼트 5

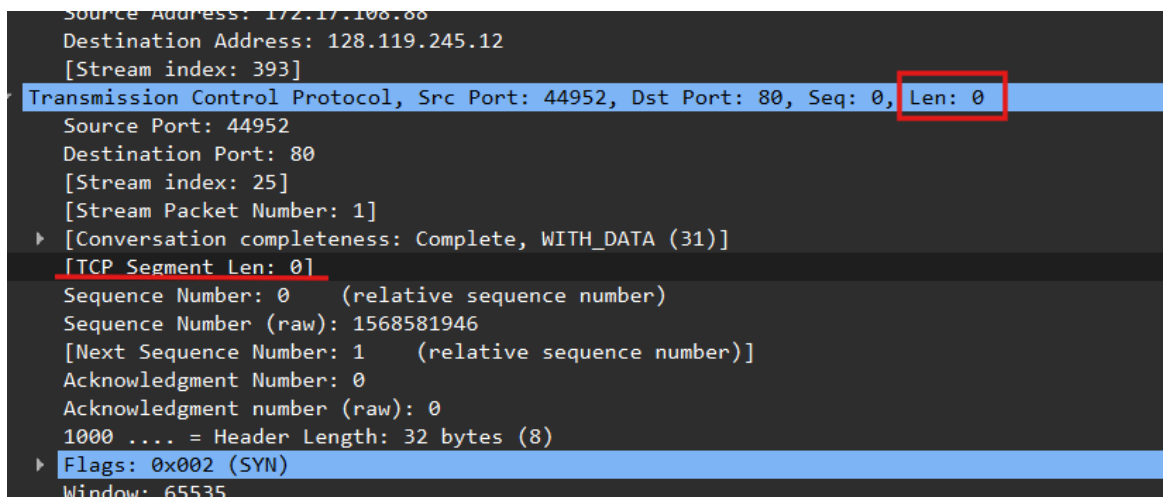
$$51.876959 - 51.675179 = 0.201780 \text{ 초}$$

##### 6. 세그먼트 6

$$51.877007 - 51.876959 = 0.000048 \text{ 초}$$

- $RTT_1 \approx 0.202686 \text{ s}$
  - $RTT_2 \approx 0.000065 \text{ s}$
  - $RTT_3 \approx 0.205117 \text{ s}$
  - $RTT_4 \approx 0.204715 \text{ s}$
  - $RTT_5 \approx 0.201780 \text{ s}$
  - $RTT_6 \approx 0.000048 \text{ s}$
- Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the [gaia.cs.umass.edu](http://gaia.cs.umass.edu) server. Then select: Statistics→TCP Stream Graph→Round Trip Time Graph.

#### 1. What is the length of each of the first six TCP segments?



```
Source Address: 172.17.108.88
Destination Address: 128.119.245.12
[Stream index: 393]
Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0
Source Port: 44952
Destination Port: 80
[Stream index: 25]
[Stream Packet Number: 1]
▶ [Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 1568581946
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 .... = Header Length: 32 bytes (8)
▶ Flags: 0x002 (SYN)
Window: 65535
```

TCP 페이로드(payload) → IP/TCP 헤더 제외한 실제 데이터 길이, 단위는 항상 byte  
단위: 바이

1. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0



- [TCP Segment Len: 0]
2. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 0, Ack: 1, Len: 0  
→ [TCP Segment Len: 0]
  3. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 0  
→ [TCP Segment Len: 0]
  4. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 615  
→ [TCP Segment Len: 615]
  5. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 616, Ack: 1, Len: 13140  
→ [TCP Segment Len: 13140]
  6. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 1, Ack: 616, Len: 0  
→ [TCP Segment Len: 0]
2. What is the minimum amount of available buffer space advertised at the receiver for the entire trace? Does the lack of receiver buffer space ever throttle(차단 혹은 전송 rate 감소) the sender?

---

TCP의 윈도우 : 한번에 보낼 수 있도록 수신자가 허용하는 데이터 양.

수신 버퍼의 남은 공간을 공시(advertise

⇒ 송신자는 이 값을 넘도록 보낼 수 없다.

**advertised = Window Size Value × Window Scaling Factor**

---

1. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 0, Len: 0
  - Window: 65535
  - [Calculated window size: 65535]
 → 65535 (scaling 없음)
2. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 0, Ack: 1, Len: 0

- Window: 64240
  - [Calculated window size: 64240]
- 64240 (scaling 없음)
3. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 0
- Window: 255
  - [Calculated window size: 65280]
  - [Window size scaling factor: 256]
- $255 \times 256 = \mathbf{65280}$
4. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 1, Ack: 1, Len: 615
- Window: 255
  - [Calculated window size: 65280]
  - [Window size scaling factor: 256]
- $255 \times 256 = \mathbf{65280}$
5. Transmission Control Protocol, Src Port: 44952, Dst Port: 80, Seq: 616, Ack: 1, Len: 13140
- Window: 255
  - [Calculated window size: 65280]
  - [Window size scaling factor: 256]
- $255 \times 256 = \mathbf{65280}$
6. Transmission Control Protocol, Src Port: 80, Dst Port: 44952, Seq: 1, Ack: 616, Len: 0
- Window: 498
  - [Calculated window size: 63744]
  - [Window size scaling factor: 128]
- $498 \times 128 = \mathbf{63744}$
- 
- 클라이언트 최소 advertised window = 65280 bytes

- 서버 최소 advertised window = 63744 bytes

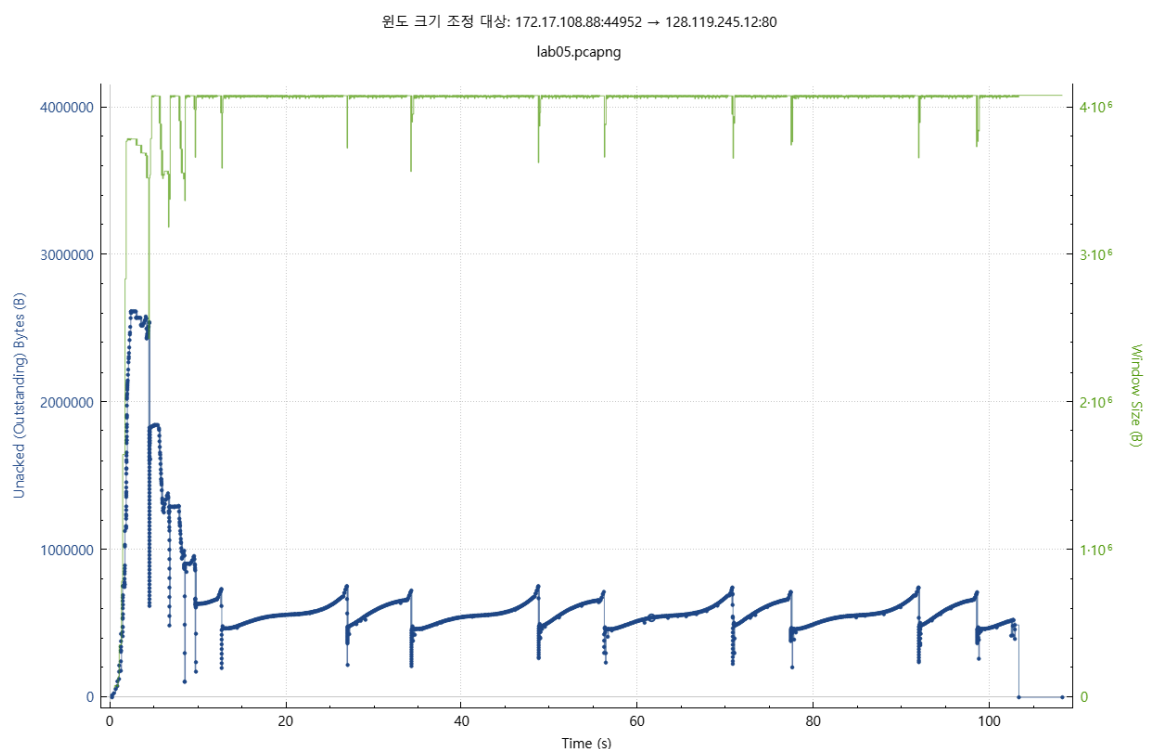
서버의 advertised window:

- 최초 SYN/ACK: **64240**
- 이후 cumulative ACK: **63744**

→ 단 496바이트 감소, buffer\_used 증가 ⇒ buffer\_free 감소로 인한 정상범위 내 감소량.

수신 버퍼 부족으로 인한 throttling으로 보기는 어렵다.

클라이언트 → 서버 방향의 TCP 세그먼트에서 윈도우 크기(Window size) 조절 통계



- 녹색 곡선 = advertised receive window(rwnd)가 여러 지점에서 급락.
- 그 시점마다 sender의 unacked bytes(또는 bytes-in-flight)가 더 이상 증가하지 않고, rwnd 모양을 따라 같이 급락.

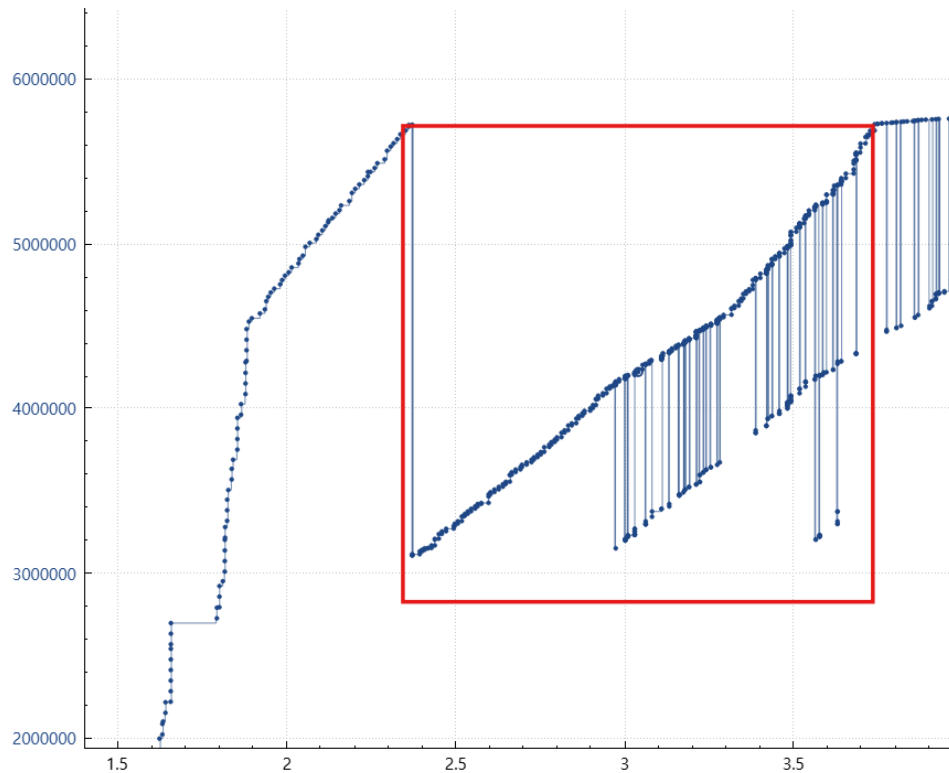
cwnd > rwnd인 상황에서, unacked bytes가 rwnd를 넘어서지 않음(rwnd로 제한 됨).

cwnd < rwnd인 상황에서, rwnd 급락 시 unacked bytes가 rwnd를 따라 같이 급락.

→ lack of receiver buffer space가 sender를 throttle함을 관찰함.

3. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

Sequence Number 그래프(TCP Sequence → Time-Sequence Graph(Stevens))로 확인



이전에 보냈던 sequence number인데 다시 보내고 있다.

→ 재전송이 일어나고 있다.

4. How much data does the receiver typically acknowledge in an ACK? Can you identify cases where the receiver is ACKing every other received segment (see the table on next page).

손실, SACK 옵션, out-of-order가 없으며 payload가 연속적으로 증가하는 구간 선택

|                                                 |                                                                                                |
|-------------------------------------------------|------------------------------------------------------------------------------------------------|
| 5175 49.823092 172.17.108.88 34.120.195.249 TCP | 55 [TCP Keep-Alive] 47852 → 443 [ACK] Seq=2154 Ack=831 Win=64512 Len=1                         |
| 5176 49.829136 34.120.195.249 172.17.108.88 TCP | 66 [TCP Keep-Alive ACK] 443 → 47852 [ACK] Seq=831 Ack=2155 Win=267776 Len=0 SLE=2154 SRE=21    |
| 5312 51.469091 172.17.108.88 128.119.245.12 TCP | 66 44952 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM                            |
| 5339 51.671777 128.119.245.12 172.17.108.88 TCP | 66 80 → 44952 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128                 |
| 5340 51.671842 172.17.108.88 128.119.245.12 TCP | 54 44952 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0                                                |
| 5341 51.672244 172.17.108.88 128.119.245.12 TCP | 669 44952 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=615 [TCP PDU reassembled in 51109]         |
| 5342 51.675179 172.17.108.88 128.119.245.12 TCP | 13194 44952 → 80 [ACK] Seq=616 Ack=1 Win=65280 Len=13140 [TCP PDU reassembled in 51109]        |
| 5366 51.876959 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=616 Win=63744 Len=0                                              |
| 5367 51.876959 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=2076 Win=67072 Len=0                                             |
| 5368 51.876959 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=7916 Win=78464 Len=0                                             |
| 5369 51.876959 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=13756 Win=79744 Len=0                                            |
| 5370 51.877007 172.17.108.88 128.119.245.12 TCP | 27794 44952 → 80 [PSH, ACK] Seq=13756 Ack=1 Win=65280 Len=27740 [TCP PDU reassembled in 51109] |
| 5393 52.073932 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=15216 Win=82688 Len=0                                            |
| 5394 52.073932 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=21056 Win=78464 Len=0                                            |
| 5395 52.073932 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=28356 Win=78464 Len=0                                            |
| 5396 52.073932 128.119.245.12 172.17.108.88 TCP | 60 80 → 44952 [ACK] Seq=1 Ack=34106 Win=79744 Len=0                                            |

- 5366 Ack = 616

- 5367 Ack = 2076 ( $2076 - 616 = 1460$ )
- 5368 Ack = 7916 ( $7916 - 2076 = 5840$ ) ← 세그먼트 여러 개를 단번에 ACK

$\Delta\text{ACK} \neq 2920 (1460 \times 2)$

- 5369 Ack = 13756 ( $13756 - 7916 = 5840$ )
- 5393 Ack = 15216 ( $15216 - 13756 = 1460$ )
- 5394 Ack = 21056 ( $21056 - 15216 = 5840$ )

→ payload를 **TCP segmentation** 후 여러 segment를 한 번에 ACK

delayed ACK (= every-other-segment ACK)은 관찰되지 않는다.

여러 segment를 한 번에 ACK하는 누적 ACK만 관찰된다.

```
tcp.len == 0 && ip.src == 128.119.245.12 && tcp.flags.ack ==
```

로 필터 걸어 추출

전체 segment를 뒤져봐도 "delayed ACK = 2 세그먼트마다 ACK ( $\Delta\text{ACK} \approx 2920$ )" 패턴은 단 1번도 나타나지 않는다.

다른 컴퓨터로 시도해봐도 마찬가지로의 결과를 얻어 지친 나머지, 실습환경을 감안해 결론만 내보았다.

송신자가 연속적으로 많은 세그먼트를 보낼 때, 수신자는 세그먼트 1개 올 때마다 ACK 1개를 보낸다. 다만 이 경우 ACK 패킷이 너무 많아져 네트워크 부담이 증가해 비효율적이다.

따라서 2개 데이터 세그먼트를 받을 때 까지 기다렸다가 한번의 ack packet을 보내는 것이 delayed ack algorithm이다.

Ethernet MTU(1500 bytes)에서 IP header(20), TCP header(20)을 뺀 1460의 두배인

2920이 증가한 값 ACK number 시퀀스를 보이게 된다.

최대 1460의 데이터를 전송하는 데이터패킷 두개 짜 마다 2920가 증가한 ACK number를 기댓값으로 보내는 것이다.

5. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

- Try to upload files with various sizes

- Select a TCP segment in the Wireshark's "listing of captured-packets" window. Then select the menu : *Statistics*→*TCP Stream Graph*→*Time-Sequence-Graph(Stevens)*.
- Question: (Part 5/5)
  1. Use the *Time-Sequence-Graph(Stevens)* plotting tool to view the sequence number versus time plot of segments being sent from the client to the `gaia.cs.umass.edu` server.
    - Can you identify where TCP's slowstart phase begins and ends, and where congestion avoidance takes over? Comment on ways in which the measured data differs from the idealized behavior of TCP that we've studied in the text.
    - Answer the questions above for the trace that you have gathered when you transferred a file from your computer to `gaia.cs.umass.edu`